

AndanteSys

Tanjil Shahid Khan N°2022189

Licenciatura Engenharia Informática

P-II



Índice

Introdução	2
Problemas	2
Objetivos	3
Estado da Arte	3
Metodologia	3
Modelo ER	4
Entidades	4
Regras de Negócio	5
Modelo de Classes	5
Desenvolvimento	9
Views	9
O Menu Principal de Entrada	9
Janela do Modo Administrador	10
Janela do Modo Cliente	11
Models	15
Modelo Pessoa	15
Modelo CartaoAndante	16
Modelo AndanteAzul	17
Modelo AndanteGold	17
Modelo Zona	18
Modelo linha	20
Helpers	22
PessoaHelper	23
CartaoHelper	23
RegistoValidacaoHelper	24
ZonaHelper	25
EstacaoHelper	26
Janela Administrador	26
Gestão de linhas, zonas e estações	28
Gestão de cartões	31
Visualização de registos	39
Janela Cliente	41
Autenticação	41
Ver zonas e estações	43
Validar estações	44
Conclusão	46

Introdução

No âmbito da unidade curricular de P-II, o desenvolvimento de um simulador de bilhética para a rede de transportes públicos da Área Metropolitana do Porto, especificamente o Metro do Porto, surge como uma oportunidade ideal para aplicar conceitos avançados de modelação de software, replicando regras de negócio complexas do mundo real através de código robusto, escalável e tipificado em C#. Neste relatório apresenta, numa primeira fase, a especificação dos requisitos e o desenho da arquitetura do sistema através de diagramas UML (Unified Modeling Language), com especial foco no Modelo de Classes e nas suas relações de herança e cardinalidade. Posteriormente, é detalhado o desenvolvimento prático do código em C#, demonstrando a implementação das regras de negócio, a aplicação de conceitos de Polimorfismo e Encapsulamento, e a metodologia utilizada para testar e apurar os resultados finais da simulação. Meti o projeto num repositório no github para conseguir dar track nas tarefas

<https://github.com/tskxz/andante-sys>

Problemas

Os sistemas de bilhética enfrentam desafios lógicos exigentes que a modelação tradicional falha em resolver de forma simples. Os principais problemas a abordar neste projeto são:

- Controlo Temporal de Transbordos: Validar as regras de transbordo (troca de linha), permitindo que o passageiro mude de composição sem que lhe seja cobrada uma nova viagem, desde que esteja dentro do tempo limite estipulado para aquela assinatura (ex: 1h para Z2, 1h15m para Z4).
- Incompatibilidade de Títulos: Impedir a coexistência de títulos de viagem de zonas diferentes no mesmo suporte físico ocasional (Cartão Azul), o que exige uma validação estrita no momento do carregamento.

Objetivos

O objetivo geral é desenvolver uma aplicação em C# que simula o ecossistema de validação do Metro do Porto.

Os objetivos específicos incluem:

- Modelar e distinguir os comportamentos dos cartões Andante Azul (títulos ocasionais) e Andante Gold (assinaturas mensais) através de herança e polimorfismo.
- Simular os Validadores das estações, capazes de ler os cartões, verificar a validade do título para a zona atual e gerir o tempo restante de viagem.
- Criar eventos que simule a viagem de um passageiro passando por múltiplas estações e linhas.

Estado da Arte

Atualmente, os sistemas de bilhética evoluíram de cartões magnéticos simples para sistemas baseados em NFC (Near Field Communication) e soluções Account-Based Ticketing (como a aplicação Anda do Metro do Porto). Tecnicamente, estes sistemas utilizam arquiteturas orientadas a objetos na sua programação base para garantir a segurança dos dados e o encapsulamento do saldo. Em ambiente académico, o uso de C# é a escolha padrão da indústria para simular estes ambientes devido à sua tipagem forte e gestão eficiente de memória, superando linguagens procedimentais na modelação de entidades físicas como "Validadores", "Estações" e "Cartões".

Metodologia

O projeto será desenvolvido seguindo uma metodologia ágil dividida nas seguintes fases:

- Fase de Análise: Levantamento das regras oficiais do Andante (tabela de tempos por zona e regras de transbordo).

- Fase de Design (UML): Criação do Diagrama de Classes detalhado, especificando modificadores de acesso, atributos, métodos e relações.
- Fase de Implementação: Codificação em C# no IDE Visual Studio focando primeiro na estrutura base das classes e depois na lógica dos métodos de validação.
- Fase de Testes: Criação de cenários de teste específicos

Modelo ER

Antes de começar a desenvolver um programa, convém planear e fazer modelo Entidade-Relacionamento e diagramas para estruturar o projeto.

Quis fazer primeiro o modelo ER porque utilizei a estratégia de mapeamento de herança chamada TPT (Table-Per-Type). A Programação Orientada a Objetos conecta-se muito com o armazenamento de dados. Assim consigo facilmente fazer o modelo e diagrama de classes usando UML. Decidi escrever num bloco de notas e começar a escrever o meu planeamento. Aqui está as entidades e relacionamentos:

Entidades

```
CARTAO: IdCartao, DataEmissao, TipoCartao
CARTAO_AZUL: IdCartao, SaldoViagens, ZonaContratada
CARTAO_GOLD: IdCartao, NomeTitular, NumDocumento, MesPago
ESTACAO: IdEstacao, NomeEstacao, ZonaAndante
VALIDADOR: IdValidador, IdEstacao
REGISTO_VALIDACAO: IdValidacao, IdCartao, IdValidador, DataHora, Resultado
```

Regras de Negócio

Relacionamentos e Cardinalidades (Regras de Negócio)

CARTAO para CARTAO_AZUL / CARTAO_GOLD (1:1)

Um CARTAO pode ser especificamente 1 CARTAO_AZUL ou 1 CARTAO_GOLD.

ESTACAO para VALIDADOR (1:N)

Uma ESTACAO possui N de VALIDADORES instalados. Um VALIDADOR pertence obrigatoriamente a 1 única ESTACAO.

CARTAO para REGISTO_VALIDACAO (1:N)

Um CARTAO pode ter N de REGISTO_VALIDACAO ao longo do tempo. Cada REGISTO_VALIDACAO pertence obrigatoriamente a 1 único CARTAO.

VALIDADOR para REGISTO_VALIDACAO (1:N)

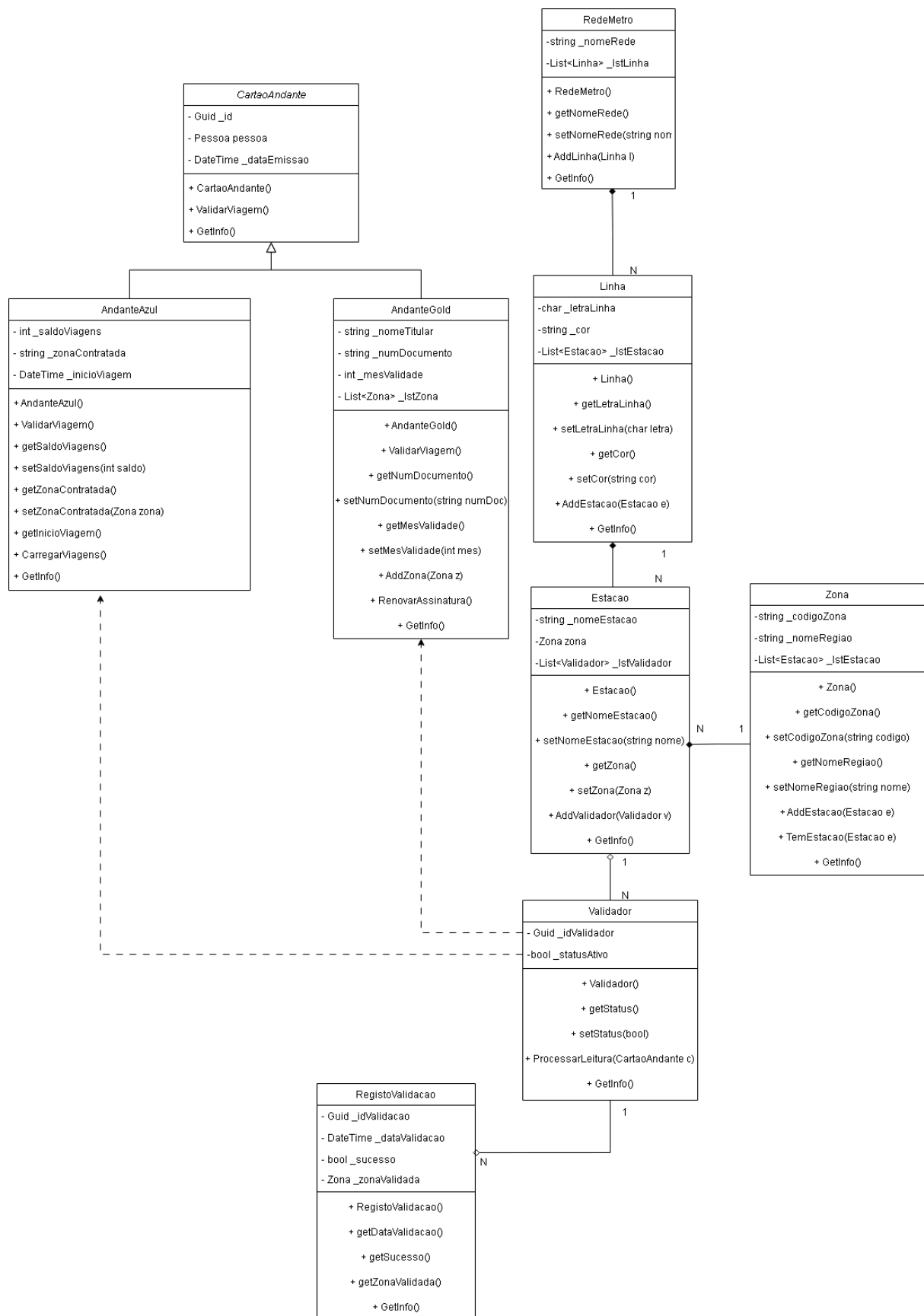
Um VALIDADOR regista N eventos de REGISTO_VALIDACAO. Cada REGISTO_VALIDACAO ocorre em 1 único VALIDADOR.

Modelo de Classes

Fiz o diagrama de classes no site draw.io. O modelo representa o funcionamento de um sistema de validação de viagens do Metro utilizando cartões Andante. A estrutura foi concebida segundo os princípios da programação orientada a objetos, recorrendo a conceitos como herança, associação e composição para modelar as diferentes entidades do sistema.

A classe abstrata CartaoAndante define as características e comportamentos comuns a todos os cartões, servindo de superclasse para os tipos específicos AndanteAzul e AndanteGold. Esta abordagem permite reutilizar funcionalidades comuns e especializar o comportamento de validação de acordo com as características de cada tipo de cartão.

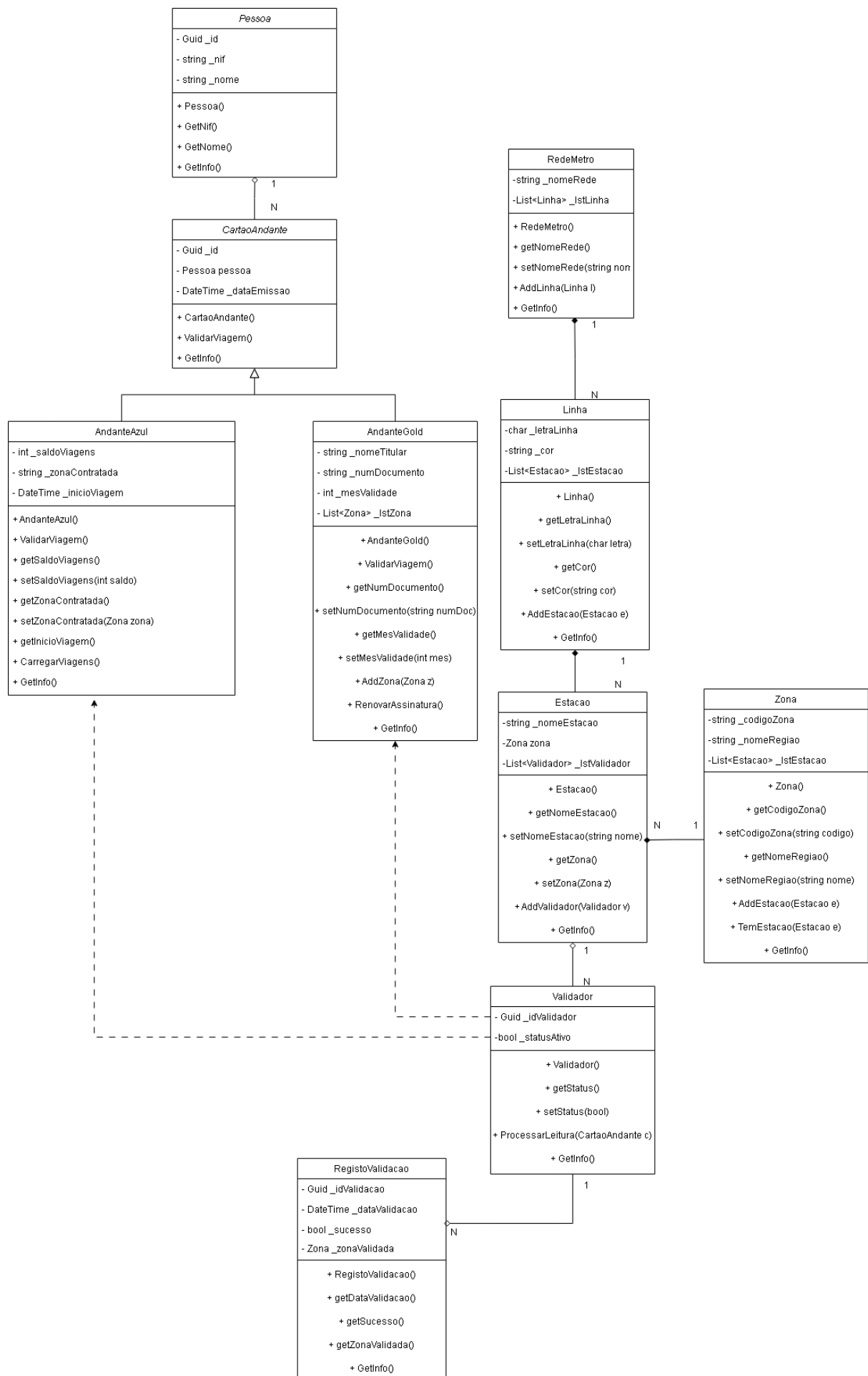
A infraestrutura da rede de metro é representada pelas classes RedeMetro, Linha, Estacao e Validador, organizadas hierarquicamente. A classe RedeMetro agrega várias linhas, cada linha é constituída por várias estações e cada estação possui um conjunto de validadores responsáveis pela leitura dos cartões e processamento das validações. Para registar o histórico das operações efetuadas, foi criada a classe RegistoValidacao, que armazena informação sobre cada validação realizada, incluindo a data e hora, o resultado da operação e a zona validada. Desta forma, é possível manter um registo detalhado da utilização do sistema.



Esqueci-me de um pormenor muito importante, a pessoa tem relacionamento com o cartão. Uma pessoa pode possuir N de cartões. Um cartão pode estar associado a no máximo 1 pessoa.

Se o cartão for do tipo cartão gold, o relacionamento é obrigatório (1:1) para identificar o titular.

Se for do tipo cartão azul, o relacionamento é inexistente (1:0), mantendo o bilhete estritamente anónimo. Acrescentei a classe pessoa no diagrama de classes UML.



Desenvolvimento

Views

O Menu Principal de Entrada

O utilizador inicia o simulador e depara-se com um menu de escolha de perfil:

- * Opção 1: Área do Administrador (Backoffice / Gestão de Dados)
- * Opção 2: Área do Cliente (Frontoffice / Viagem e Validação)
- * Opção 3: Fechar o Simulador

Na interface, criei os 3 botões

```
<Button x:Name="btnAdmin"
        Content="Área do Administrador (Backoffice)"
        Height="50"
        Margin="0,0,0,15"
        FontSize="14"
        FontWeight="SemiBold"
        Background="#007ACC"
        Foreground="White"
        Cursor="Hand"
        Click="BtnAdmin_Click"/>

<Button x:Name="btnCliente"
        Content="Área do Cliente (Frontoffice)"
        Height="50"
        Margin="0,0,0,30"
        FontSize="14"
        FontWeight="SemiBold"
        Background="#28A745"
        Foreground="White"
        Cursor="Hand"
        Click="BtnCliente_Click"/>

<Button x:Name="btnSair"
        Content="Fechar o Simulador"
        Height="40"
        Width="180"
        FontSize="12"
        Background="#DC3545"
        Foreground="White"
        Cursor="Hand"
        HorizontalAlignment="Center"
        Click="BtnSair_Click"/>
```

Para já está assim

```
1 reference
private void BtnAdmin_Click(object sender, RoutedEventArgs e)
{
}

1 reference
private void BtnCliente_Click(object sender, RoutedEventArgs e)
{
}

1 reference
private void BtnSair_Click(object sender, RoutedEventArgs e)
{
    this.Close();
}
```

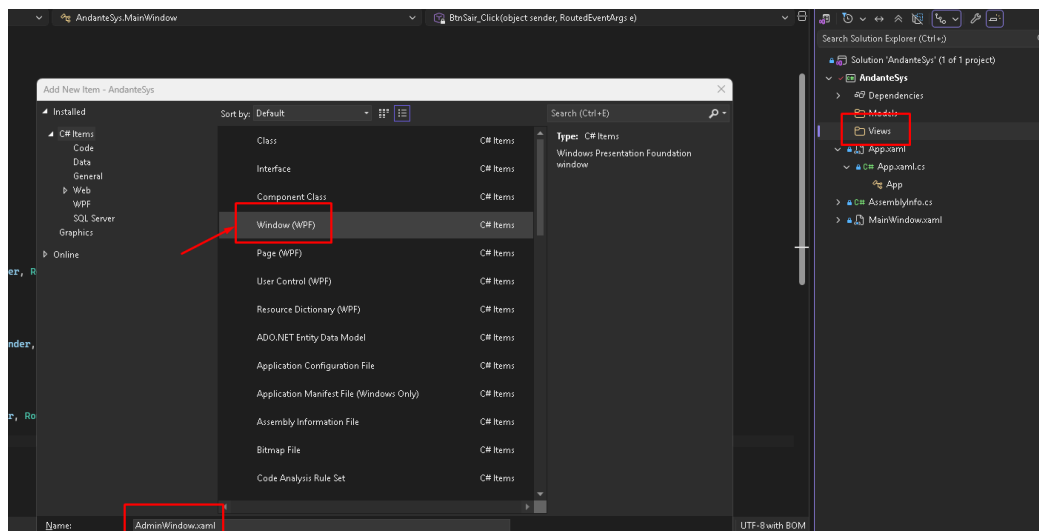
Depois irei implementar as funcionalidades para ir para a janela de administrador / cliente.



Janela do Modo Administrador

O objetivo desta área é alimentar o sistema simulado com clientes e cartões.

Como vou criar uma nova janela, irei criar uma pasta chamada Views e adicionar um novo item WPF chamado AdminWindow.



Dei o nome de BtnVoltar_Click

```
<Border Background="White" CornerRadius="5" Padding="15">
  <StackPanel>
    <TextBlock Text="Gestão de Títulos" FontSize="18" FontWeight="Bold" Foreground="#007ACC" Margin="0,0,15"/>
    <Button x:Name="btnVoltar" Content="← Voltar ao Menu" Height="30" Background="#6C757D" Foreground="White" Click="BtnVoltar_Click" Margin="0,0,20"/>
    <TextBlock Text="Os controlos de criação vão ficar aqui..." VerticalAlignment="Center" HorizontalAlignment="Center" Foreground="Gray" Margin="0,100,0,0"/>
  </StackPanel>
</Border>

<Grid.Column="1" Margin="15,0,0,0">
  <Border Background="White" CornerRadius="5" Padding="15">
    <StackPanel>
      <TextBlock Text="Cartões" FontSize="18" FontWeight="Bold" Foreground="#333333" Margin="0,0,15"/>
      <TextBlock Text="A tabela (DataGrid) vai ficar deste lado..." VerticalAlignment="Center" HorizontalAlignment="Center" Foreground="Gray" Margin="0,150,0,0"/>
    </StackPanel>
  </Border>
</Grid>
```

Fui para o AdminWindow.xaml.cs e se clicar no botão voltar, mostro o menu principal

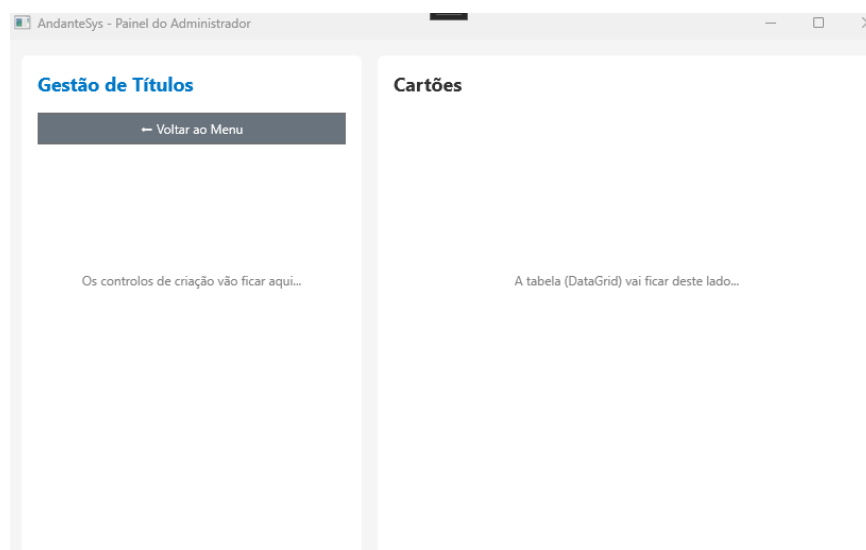
```
// fechar a janela e reabrir o menu principal
0 references
private void BtnVoltar_Click(object sender, RoutedEventArgs e)
{
    // janela principal main window
    MainWindow mainWin = new MainWindow();
    mainWin.Show();

    this.Close();
}
```

Volto para o código do UI do menu principal e implementei a funcionalidade de mostrar a janela do admin

```
private void BtnAdmin_Click(object sender, RoutedEventArgs e)
{
    AndanteSys.Views.AdminWindow adminWin = new AndanteSys.Views.AdminWindow();
    adminWin.Show();
    this.Close();
}
```

Isto é um esqueleto que depois irei implementar a gestão de títulos e de cartões



Janela do Modo Cliente

O utilizador volta ao Menu Principal e escolhe a "Área do Cliente". O ecrã inicia bloqueado, com uma caixa de texto única que aceita NIF ou ID do Cartão.

Para a Área do Cliente, iremos começar bloqueada (fase de Login) e depois se transforma no painel de viagem (fase de Validação)

Para fazer isso, devo colocar dois painéis (StackPanel ou Grid) no mesmo sítio e controlar a propriedade Visibility deles através do C#.

Quando o utilizador inicia, o painel do Login está Visible (Visível) e o da Viagem está Collapsed (Escondido). Após o login com sucesso, inverte-se os papéis.

Implementei 4 funções.

```
<Button x:Name="btnLogin" Content="Simular Encosto (Login)" Height="40" Background="#28A745" Foreground="White" FontSize="14" FontWeight="Bold" Cursor="Hand" Click="BtnLogin_Click" Margin="0,0,0,15"/>
<Button x:Name="btnVoltarMenu" Content="Voltar ao Menu Inicial" Height="30" Background="#6C757D" Foreground="White" Cursor="Hand" Click="BtnVoltar_Click" />
</StackPanel>
</Border>

<Border x:Name="panelViagem" Background="White" CornerRadius="8" Padding="20" Visibility="Collapsed" BorderBrush="#DDD" BorderThickness="1">
  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <DockPanel Grid.Row="0" LastChildFill="False" Margin="0,0,0,15">
      <StackPanel DockPanel.Dock="Left">
        <Button x:Name="btnLogout" Content="Retirar Cartão (Logout)" DockPanel.Dock="Right" Height="30" Width="140" Background="#DC3545" Foreground="White" Click="BtnLogout_Click" />
      </StackPanel>
    </DockPanel>
    <Separator Grid.Row="0" VerticalAlignment="Bottom" />
    <StackPanel Grid.Row="1" Margin="0,15,0,0">
      <TextBlock Text="1. Selecione a Estação de Embarque:" FontSize="13" FontWeight="SemiBold" Margin="0,0,0,5"/>
      <ComboBox x:Name="cbEstacoes" Height="35" FontSize="13" VerticalContentAlignment="Center" Margin="0,0,20,0" />
      <Button x:Name="btnValidar" Content="VALIDAR VIAGEM" Height="45" Background="#007ACC" Foreground="White" FontSize="15" FontWeight="Bold" Cursor="Hand" Click="BtnValidar_Click" Margin="0,0,25,0"/>
    </StackPanel>
  </Grid>
</Border>
```

O botão de logOff faz com que saia da janela de viagem e mostre a janela do login.

A função voltar mostra a janela principal.

```
// esconde a viagem e mostra o login outra vez
private void BtnLogout_Click(object sender, RoutedEventArgs e)
{
    txtLoginInput.Clear();
    panelViagem.Visibility = Visibility.Collapsed;
    panelLogin.Visibility = Visibility.Visible;
}

// voltar ao menu principal
private void BtnVoltar_Click(object sender, RoutedEventArgs e)
{
    MainWindow mainWin = new MainWindow();
    mainWin.Show();
    this.Close();
}
```

Na função de login, ao entrar com o utilizador nif 123456789, simulei que é o carlos silva e tem um cartão com zona PRT1, também posso entrar com cartão azul que tem 2 viagens. Após entrar

```
private void BtnLogin_Click(object sender, RoutedEventArgs e)
{
    string input = txtLoginInput.Text.Trim();
    if(input.Length == 0)
    {
        MessageBox.Show("Por favor, introduza um NIF ou ID válido.", "Aviso", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // TODO: isto é uma solução temporária, dps mudo isto, tenho que criar as classes do Model
    if(input == "123456789")
    {
        lblStatusSessao.Text = "Sessão Ativa: Carlos Silva (Passe Gold)";
        lblSaldoSessao.Text = "Assinatura Ativa: Zona PRT1";
    } else if (input.ToUpper() == "AZUL-01")
    {
        lblStatusSessao.Text = "Sessão Ativa: Cartão Ocasional Anónimo (ID: AZUL-01)";
        lblSaldoSessao.Text = "Saldo Atual: 2 Viagens";
    } else
    {
        // para já aceita tudo so pa testar o layout
        lblStatusSessao.Text = $"Sessão Ativa: Cartão de Teste ({input})";
        lblSaldoSessao.Text = "Modo de simulação geral.";
    }

    // esconder o login e mostrar viagem
    panelLogin.Visibility = Visibility.Collapsed;
    panelViagem.Visibility = Visibility.Visible;

    // Reseta o validador para o estado inicial
    ellipseLuz.Fill = Brushes.LightGray;
    lblDisplayValidador.Text = "AGUARDANDO VALIDAÇÃO...";
}
}
```

Isto seria uma solução temporária, simulamos o comportamento do programa em que valida as viagens. Ao entrar o utilizador com nif 123456789, se validarmos o cartão com a estação da trindade, vai ser validado, se não, fica inválido

```
private void BtnValidar_Click(object sender, RoutedEventArgs e)
{
    string input = txtLoginInput.Text.Trim();
    int estacaoSelecionada = cbEstacoes.SelectedIndex; // 0 = Trindade, 1 = Matosinhos
    if (input == "123456789")
    {
        if (estacaoSelecionada == 0)
        {
            ellipseLuz.Fill = Brushes.Green;
            lblDisplayValidador.Text = ">>> LUZ VERDE: VALIDADO COM SUCESSO. BOA VIAGEM! <<<";
        }
        else
        {
            ellipseLuz.Fill = Brushes.Red;
            lblDisplayValidador.Text = ">>> LUZ VERMELHA: RECUSADO - ZONA NÃO AUTORIZADA! <<<";
        }
    }
    else
    {
        // temporario, simula sucesso direto apenas para ver o comportamento do ecrã
        ellipseLuz.Fill = Brushes.Green;
        lblDisplayValidador.Text = ">>> LUZ VERDE: VIAGEM DESCONTADA. BOA VIAGEM! <<<";
    }
}
}
```

Na janela principal da função BtnCliente_Click, chamo o método show do objeto clienteWin para mostrar a janela do cliente

```
private void BtnCliente_Click(object sender, RoutedEventArgs e)
{
    AndanteSys.Views.ClienteWindow clienteWin = new AndanteSys.Views.ClienteWindow();
    clienteWin.Show();
    this.Close();
}
}
```

AndanteSys - Área do Cliente

ÁREA DO PASSAGEIRO

Por favor, aproxime o seu cartão virtual.

Insira o NIF do Titular ou o ID do Cartão:

123456789

Simular Encosto (Login)

Voltar ao Menu Inicial

AndanteSys - Área do Cliente

Sessão Ativa: Carlos Silva (Passe Gold) **Retirar Cartão (Logout)**

Assinatura Ativa: Zona PRT1

1. Selecione a Estação de Embarque:

Trindade (Zona PRT1)

VALIDAR VIAGEM

AGUARDANDO VALIDAÇÃO...

AndanteSys - Área do Cliente

Sessão Ativa: Carlos Silva (Passe Gold) **Retirar Cartão (Logout)**

Assinatura Ativa: Zona PRT1

1. Selecione a Estação de Embarque:

Trindade (Zona PRT1)

VALIDAR VIAGEM

>>> LUZ VERDE: VALIDADO COM SUCESSO. BOA VIAGEM! <<<

AndanteSys - Área do Cliente

Sessão Ativa: Carlos Silva (Passe Gold) **Retirar Cartão (Logout)**

Assinatura Ativa: Zona PRT1

1. Selecione a Estação de Embarque:

Matosinhos Sul (Zona MTS1)

VALIDAR VIAGEM

>>> LUZ VERMELHA: RECUSADO - ZONA NÃO AUTORIZADA! <<<

Models

Modelo Pessoa

Criei a classe pessoa que tem 2 atributos, nif e nome, mas também acrescentei o Guid id para representar uma chave primária e única.

```
public class Pessoa
{
    private Guid _idPessoa;
    private string _nif;
    private string _nome;

    0 references
    public Guid IdPessoa
    {
        get { return _idPessoa; }
    }

    0 references
    public string NIF
    {
        get { return _nif; }
        set
        {
            _nif = value.Trim();
            if (_nif.Length == 0)
            {
                _nif = "#####";
            }
        }
    }

    0 references
    public string Nome
    {
        get { return _nome; }
        set
        {
            _nome = value.Trim();
            if (_nome.Length == 0)
            {
                _nome = "Anónimo";
            }
        }
    }
}
```


Modelo CartaoAndante

Usamos 'abstract' para garantir que ninguém consegue criar um "CartaoAndante" genérico,

```
public abstract class CartaoAndante
{
    private Guid _idCartao;
    private DateTime _dataEmissao;
    private string _tipoCartao;
    private Pessoa _titular;

    0 references
    public Guid IdCartao
    {
        get { return _idCartao; }
    }

    0 references
    public DateTime DataEmissao
    {
        get { return _dataEmissao; }
    }

    0 references
    public string TipoCartao
    {
        get { return _tipoCartao; }
        set
        {
            _tipoCartao = value.Trim();
            if(_tipoCartao.Length == 0)
            {
                _tipoCartao = "AZUL";
            }
        }
    }

    0 references
    public Pessoa Titular
    {
        get { return _titular; }
        set
        {
            _titular = value;
        }
    }

    0 references
    protected CartaoAndante()
    {
        _idCartao = Guid.NewGuid();
        _dataEmissao = DateTime.Now;
        _tipoCartao = "";
        _titular = new Pessoa();
    }
}
```

Acrescentei o método abstrato de validar viagem. É para obrigar as subclasses a programar as suas próprias regras de validação.

```
0 references
public abstract bool ValidarViagem(string zonaEstacaoAtual);
```

Modelo AndanteAzul

Implementei a própria regra de validação do cartão azul em que desconta a viagem se tiver saldo e se a estação atual é o da zona contratada, incluí o método de carregar viagens.

```
1 reference
public class AndanteAzul : CartaoAndante
{
    private int _saldoViagens;
    private string _zonaContratada;

    1 reference
    public override bool ValidarViagem(string zonaEstacaoAtual)
    {
        // tem de ter saldo e a estação atual tem de ser da zona contratada
        if(_saldoViagens > 0 && zonaEstacaoAtual.ToUpper() == _zonaContratada)
        {
            // descontar 1 viagem
            _saldoViagens--;
            return true;
        }
        return false;
    }

    0 references
    public void CarregarViagens(int quantidade, string zona)
    {
        if (quantidade > 0)
        {
            _saldoViagens += quantidade;
            _zonaContratada = zona;
        }
    }

    0 references
    public AndanteAzul() : base()
    {
        _saldoViagens = 0;
        _zonaContratada = "PRT1";
        TipoCartao = "AZUL";
    }
}
```

Modelo AndanteGold

Tenho o atributo de mês em que está validado e lista de zonas autorizadas.

```
private int _mesPago;
private List<Zona> _zonasAutorizadas;

0 references
public int MesPago
{
    get { return _mesPago; }
    set {
        _mesPago = value;
        if (_mesPago < 1 || _mesPago > 12)
        {
            _mesPago = 0;
        }
    }
}

0 references
public List<Zona> ZonasAutorizadas
{
    get { return _zonasAutorizadas; }
    set { _zonasAutorizadas = value; }
}
```

Para validar as viagens, o mês validado tem que coincidir com o atual e qualquer uma das zonas que está na lista de autorizadas, tem que coincidir com a zona atual

na altura do momento em que valido naquela estação. A estação pertence a uma zona. Também inclui o método de adicionar zona autorizada e renovar assinatura

```
2 references
public override bool ValidarViagem(Zona zonaEstacaoAtual)
{
    int mesAtual = DateTime.Now.Month;
    // o mes do passe tem de estar atualizado e a zona da estação tem de estar autorizada
    if (_mesPago == mesAtual)
    {
        if (zonaEstacaoAtual != null)
        {
            if (_zonasAutorizadas.Any(z => z.CodigoZona == zonaEstacaoAtual.CodigoZona))
            {
                return true;
            }
        }
    }
    return false;
}

0 references
public void AddZonaAutorizada(Zona zona)
{
    if (zona == null)
        return;

    string codigo = zona.CodigoZona?.Trim().ToUpper();
    if (string.IsNullOrEmpty(codigo))
        return;

    if (!_zonasAutorizadas.Any(z => z.CodigoZona == codigo))
    {
        Zona nova = new Zona { CodigoZona = codigo };
        _zonasAutorizadas.Add(nova);
    }
}
```

```
0 references
public void RenovarAssinatura(int mes)
{
    _mesPago = mes;
}

0 references
public AndanteGold()
{
    _mesPago = 0;
    _zonasAutorizadas = new List<Zona> { new Zona { CodigoZona = "PRT1" }, new Zona { CodigoZona = "PRT2" } };
    TipoCartao = "GOLD";
}

3 references
public override string ToString()
{
    return $"{base.ToString()} | Mês Pago: {_mesPago}, Zonas Autorizadas: {string.Join(", ", _zonasAutorizadas.Select(z => z.CodigoZona))}";
}
```

Modelo Zona

O modelo zona tem os atributos código, nome da região e as estações que pertencem a zona. Temos o método de adicionar estação para a lista de estações em que vai nos ser útil para os processos de validação dos cartões andantes.

```

14 references
public class Zona
{
    private string _codigoZona;
    private string _nomeRegiao;
    private List<Estacao> _lstEstacao;

    11 references
    public string CodigoZona
    {
        get { return _codigoZona; }
        set {
            _codigoZona = value.Trim().ToUpper();
            if(_codigoZona.Length == 0)
            {
                _codigoZona = "???";
            }
        }
    }

    0 references
    public string NomeRegiao
    {
        get { return _nomeRegiao; }
        set {
            _nomeRegiao = value.Trim();
            if(_nomeRegiao.Length == 0)
            {
                _nomeRegiao = "Sem Região";
            }
        }
    }
}

```

```

0 references
public List<Estacao> LstEstacao
{
    get { return _lstEstacao; }
}

3 references
public Zona()
{
    _codigoZona = "";
    _nomeRegiao = "";
    _lstEstacao = new List<Estacao>();
}

0 references
public void AddEstacao(Estacao e)
{
    if (e != null && !_lstEstacao.Contains(e))
    {
        _lstEstacao.Add(e);
    }
}

0 references
public bool TemEstacao(Estacao e)
{
    return _lstEstacao.Contains(e);
}

0 references
public override string ToString()
{
    return $"Zona: {_codigoZona} ({_nomeRegiao}) - {_lstEstacao.Count} estações vinculadas";
}
}

```

Modelo Estação

Uma estação tem o nome, a zona que pertence e vários validadores. Num validador, verificamos a zona a partir da estação em que está o validador.

```

12 references
public class Estacao
{
    private string _nomeEstacao;
    private Zona _zona;
    private List<Validador> _lstValidador;

    1 reference
    public string NomeEstacao
    {
        get { return _nomeEstacao; }
        set { _nomeEstacao = value.Trim(); }
    }

    3 references
    public Zona Zona
    {
        get { return _zona; }
        set { _zona = value; }
    }

    0 references
    public List<Validador> LstValidador
    {
        get { return _lstValidador; }
    }

    0 references
    public Estacao()
    {
        _nomeEstacao = "Estação Genérica";
        _zona = null;
        _lstValidador = new List<Validador>();
    }
}

```

```

0 references
public void AddValidador(Validador v)
{
    if (v != null && !_lstValidador.Contains(v))
    {
        _lstValidador.Add(v);
    }
}

0 references
public override string ToString()
{
    string codZona = (_zona != null) ? _zona.CodigoZona : "Nenhuma";
    return $"Estação: {_nomeEstacao} [Zona: {codZona}] - {_lstValidador.Count} Validadores";
}
}

```

Modelo linha

As linhas têm a sua letra, cor e cada linha tem várias estações. A Linha vai nos servir de propósito para agrupar as estações de forma lógica, para facilitar aos utilizadores quando estivermos a usar e interagir com o nosso sistema.

```
public class Linha
{
    private char _letraLinha;
    private string _cor;
    private List<Estacao> _lstEstacao;

    0 references
    public char LetraLinha
    {
        get { return _letraLinha; }
        set { _letraLinha = value; }
    }

    0 references
    public string Cor
    {
        get { return _cor; }
        set { _cor = value.Trim(); }
    }

    0 references
    public List<Estacao> LstEstacao
    {
        get { return _lstEstacao; }
    }

    0 references
    public Linha()
    {
        _letraLinha = '?';
        _cor = "Transparente";
        _lstEstacao = new List<Estacao>();
    }
}
```

```
0 references
public void AddEstacao(Estacao e)
{
    if (e != null && !_lstEstacao.Contains(e))
    {
        _lstEstacao.Add(e);
    }
}

0 references
public override string ToString()
{
    return $"Linha {_letraLinha} ({_cor}) - {_lstEstacao.Count} estações na rota.";
}
}
```

Modelo RedeMetro

A classe RedeMetro é o topo de toda a hierarquia do nosso sistema. A classe possui o nome da rede que no nosso caso é o Metro do Porto e tem o conjunto de linhas.

```

0 references
public class RedeMetro
{
    private string _nomeRede;
    private List<Linha> _lstLinha;

    0 references
    public string NomeRede
    {
        get { return _nomeRede; }
        set { _nomeRede = value.Trim(); }
    }

    0 references
    public List<Linha> LstLinha
    {
        get { return _lstLinha; }
    }

    0 references
    public RedeMetro()
    {
        _nomeRede = "Rede Desconhecida";
        _lstLinha = new List<Linha>();
    }

    0 references
    public void AddLinha(Linha l)
    {
        if (l != null && !_lstLinha.Contains(l))
        {
            _lstLinha.Add(l);
        }
    }
}

```

Helpers

Depois de criar os modelos das classes a orientação de objetos, iremos preencher alguns dados para o projeto. No App.xaml.cs criei as listas estáticas globais onde o sistema vai guardar os dados

```

3 references
public partial class App : Application
{
    public static List<Pessoa> lstPessoas = new List<Pessoa>();
    public static List<CartaoAndante> lstCartoes = new List<CartaoAndante>();
    public static List<Zona> lstZonas = new List<Zona>();
    public static List<Linha> lstLinhas = new List<Linha>();
    public static List<RegistoValidacao> lstRegistos = new List<RegistoValidacao>();

    0 references
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        InicializarCenarioMetro();
    }

    1 reference
    private void InicializarCenarioMetro()
    {
        // Criar duas zonas base
        Zona z1 = new Zona {CodigoZona = "PRT1", NomeRegiao = "Porto Centro" };
        Zona z2 = new Zona {CodigoZona = "MTS1", NomeRegiao = "Matosinhos" };
        lstZonas.Add(z1);
        lstZonas.Add(z2);

        // criar estacoes e validadores
        Estacao trindade = new Estacao { NomeEstacao = "Trindade", Zona = z1 };
        trindade.AddValidador(new Validador { Estacao = trindade });
        z1.AddEstacao(trindade);

        Estacao matosinhos = new Estacao { NomeEstacao = "Matosinhos Sul", Zona = z2 };
        matosinhos.AddValidador(new Validador { Estacao = matosinhos });
        z2.AddEstacao(matosinhos);

        // criar uma linha
        Linha linhaAzul = new Linha { LetraLinha = 'A', NomeLinha = "Linha de Matosinhos", Cor = "Azul" };
        linhaAzul.AddEstacao(matosinhos);
        linhaAzul.AddEstacao(trindade);
        lstLinhas.Add(linhaAzul);
    }
}

```

PessoaHelper

Como o meu projeto envolve a gestão de pessoas e a emissão de cartões, irei criar os Helpers correspondentes para manipular aquelas listas centrais do App que meti no app.xaml.cs para guardar os dados.

```
0 references
public class PessoaHelper
{
    0 references
    public void Insert(Pessoa pessoa)
    {
        if (pessoa != null)
        {
            App.lstPessoas.Add(pessoa);
        }
    }

    0 references
    public void Atualizar(Pessoa pessoa)
    {
        var pessoaExistente = App.lstPessoas.FirstOrDefault(p => p.IdPessoa == pessoa.IdPessoa);
        if (pessoaExistente != null)
        {
            pessoaExistente.Nome = pessoa.Nome;
            pessoaExistente.NIF = pessoa.NIF;
        }
    }

    0 references
    public void Apagar(Pessoa pessoa)
    {
        var pessoaExistente = App.lstPessoas.FirstOrDefault(p => p.IdPessoa == pessoa.IdPessoa);
        if (pessoaExistente != null)
        {
            App.lstPessoas.Remove(pessoaExistente);
        }
    }

    0 references
    public List<Pessoa> ListarTodas()
    {
        return App.lstPessoas;
    }
}
```

CartaoHelper

Ao implementar o cartão helper, uma coisa que gostava de alertar, com os cartões andante, não meti o método Atualizar porque normalmente não se cria da mesma forma por causa do polimorfismo e das regras de negócio.

Um CartaoAndante é uma classe abstrata com regras vivas. No mundo real, eu não "atualizo" o nome do cartão; as atualizações de um cartão são ações específicas do negócio:

Se for Azul: Atualizar significa Carregar Viagens

Se for Gold: Atualizar significa Renovar a Assinatura


```
namespace AndanteSys.Helpers
{
    0 references
    public class CartaoHelper
    {
        0 references
        public void Insert(CartaoAndante cartao)
        {
            if (cartao != null)
            {
                App.lstCartoes.Add(cartao);
            }
        }

        0 references
        public void Apagar(CartaoAndante cartao)
        {
            var cartaoExistente = App.lstCartoes.FirstOrDefault(c => c.IdCartao == cartao.IdCartao);
            if (cartaoExistente != null)
            {
                App.lstCartoes.Remove(cartaoExistente);
            }
        }

        0 references
        public List<CartaoAndante> ListarTodos()
        {
            return App.lstCartoes;
        }
    }
}
```

RegistoValidacaoHelper

Sempre que uma viagem for processada, usamos este helper para meter o rastro da operação na lista global de auditoria.

```
namespace AndanteSys.Helpers
{
    0 references
    public class RegistoValidacaoHelper
    {
        0 references
        public void Insert(RegistoValidacao registo)
        {
            if (registo != null)
            {
                App.lstRegistos.Add(registo);
            }
        }

        0 references
        public List<RegistoValidacao> ListarHistorico()
        {
            return App.lstRegistos;
        }
    }
}
```

Voltei para a classe validador e no método de processamento de leitura, uso o helper para inserir no registo

```
public bool ProcessarLeitura(CartaoAndante c)
{
    if (!_statusAtivo || c == null || _estacao == null || _estacao.Zona == null)
        return false;

    bool sucessoViagem = c.ValidarViagem(_estacao.Zona);

    RegistoValidacao novoRegisto = new RegistoValidacao();
    novoRegisto.Sucesso = sucessoViagem;
    novoRegisto.ZonaValidada = _estacao.Zona.CodigoZona;
    novoRegisto.Cartao = c;
    HistoricoGlobal.Add(novoRegisto);

    AndanteSys.Helpers.RegistoValidacaoHelper registoHelper = new AndanteSys.Helpers.RegistoValidacaoHelper();
    registoHelper.Insert(novoRegisto);
    return sucessoViagem;
}
```

ZonaHelper

```
0 references
public class ZonaHelper
{
    0 references
    public void Insert(Zona zona)
    {
        if (zona != null && !App.lstZonas.Any(z => z.CodigoZona == zona.CodigoZona))
        {
            App.lstZonas.Add(zona);
        }
    }

    0 references
    public void Apagar(Zona zona)
    {
        var zonaExistente = App.lstZonas.FirstOrDefault(z => z.CodigoZona == zona.CodigoZona);
        if (zonaExistente != null)
        {
            App.lstZonas.Remove(zonaExistente);
        }
    }

    0 references
    public List<Zona> ListarTodas()
    {
        return App.lstZonas;
    }
}
```

LinhaHelper

```
0 references
public class LinhaHelper
{
    0 references
    public void Insert(Linha linha)
    {
        if (linha != null && !App.lstLinhas.Any(l => l.LetraLinha == linha.LetraLinha))
        {
            App.lstLinhas.Add(linha);
        }
    }

    0 references
    public void Apagar(Linha linha)
    {
        var linhaExistente = App.lstLinhas.FirstOrDefault(l => l.LetraLinha == linha.LetraLinha);
        if (linhaExistente != null)
        {
            App.lstLinhas.Remove(linhaExistente);
        }
    }

    0 references
    public List<Linha> ListarTodas()
    {
        return App.lstLinhas;
    }
}
```

EstacaoHelper

As Estações pertencem a uma Linha e a uma Zona. Não costumam ficar numa lista solta na App. Por isso, o EstacaoHelper vai servir para injetar a Estação diretamente no sítio correto do sistema (adiciona à lista da Linha e à lista da Zona correspondente).

```
namespace AndanteSys.Helpers
{
    0 references
    public class EstacaoHelper
    {
        0 references
        public void Insert(Estacao estacao, Linha linhaAlvo, Zona zonaAlvo)
        {
            if (estacao == null || linhaAlvo == null || zonaAlvo == null) return;

            // vincular a estacao a sua zona
            estacao.Zona = zonaAlvo;
            zonaAlvo.AddEstacao(estacao);

            // adicionar a estacao a rota da linha selecionada
            linhaAlvo.AddEstacao(estacao);

            // instalar automaticamente o validador físico na estação
            estacao.AddValidador(new Validador { Estacao = estacao });
        }

        0 references
        public List<Estacao> ListarTodasDoSistema()
        {
            // vai buscar todas as estações registadas em todas as linhas
            return App.lstLinhas.SelectMany(l => l.lstEstacao).Distinct().ToList();
        }
    }
}
```

Janela Administrador

Alterou-se a interface gráfica da janela administrador para gerir a emissão de cartões e configurar as zonas, linhas e estações. Incluí também o balcão de atendimento que seria uma zona em baixo que serve para eu clicar num cartão da lista e escolher quantas viagens quero carregar ou se quero ativar a mensalidade.

The screenshot shows the 'Janela Administrador' interface. On the left, there's a sidebar with a 'Voltar ao Menu Principal' button and a menu with 'Emitir Cartão', 'Criar Zona', 'Criar Linha', and 'Criar Estação'. The main area is divided into two panels. The left panel, titled 'Emitir Novo Título', contains a form with fields for 'Nome do Titular', 'NIF', and 'Tipo de Cartão' (set to 'Andante Azul (Ocasional)'), and a green 'Gravar e Emitir' button. The right panel, titled 'Base de Títulos Emitidos (Fiscalização)', contains a table with columns 'ID Cartão', 'Titular', 'NIF', and 'Tipo'. Below the table, there's a section for 'Operações de Carregamento / Balcão' with a dropdown for 'Qtd. Viagens (Azul):' set to '1', a dropdown for 'Zona Contratada:', and two buttons: 'Carregar Azul' and 'Ativar Mês (Gold)'.

Adicionei a organização por Abas, usei o TabControl: O administrador agora escolhe lá em cima se quer criar um Cartão, uma Zona, uma Linha ou uma Estação de forma limpa.

Emitir Cartão | Criar Zona | Criar Linha | Criar Estação

Configurar Nova Zona

Código da Zona (Ex: PRT1, MTS1):

Nome da Região / Localidade:

Adicionar Zona Tarifária

Emitir Cartão | Criar Zona | Criar Linha | Criar Estação

Adicionar Nova Linha de Metro

Letra Identificadora (Ex: A, B, C):

Nome da Linha:

Cor Representativa:

Registar Linha na Rede

Instalar Estação e Validador

Nome da Estação:

Associar à Linha:

Pertence à Zona Tarifária:

Instalar Estação no Sistema

Gestão de linhas, zonas e estações

Implementei a criação de linhas em que busca a partir dos textBoxs e uso o helper para inserir os dados.

```
private void BtnCriarLinha_Click(object sender, RoutedEventArgs e)
{
    string letra = txtLetraLinha.Text.Trim().ToUpper();
    string nome = txtNomeLinha.Text.Trim();
    string cor = txtCorLinha.Text.Trim();

    if (letra.Length == 0)
    {
        MessageBox.Show("Informe a letra identificadora da linha.", "Erro", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var linha = new AndanteSys.Models.Linha { LetraLinha = letra[0], NomeLinha = nome, Cor = cor };
    var lh = new Helpers.LinhaHelper();
    lh.Insert(linha);

    LoadLinesIntoCombo();
    MessageBox.Show("Linha registada.", "OK", MessageBoxButton.OK, MessageBoxImage.Information);
}
```

De seguida, implementei a criação de zonas em que pego os dados através do textbox e uso o helper da zona para inserir o dado.

```
private void BtnCriarZona_Click(object sender, RoutedEventArgs e)
{
    string codigo = txtCodigoZona.Text.Trim().ToUpper();
    string nome = txtNomeRegiao.Text.Trim();

    if (codigo.Length == 0)
    {
        MessageBox.Show("Informe o código da zona.", "Erro", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var zona = new AndanteSys.Models.Zona { CodigoZona = codigo, NomeRegiao = nome };
    var zh = new Helpers.ZonaHelper();
    zh.Insert(zona);

    LoadZonesIntoCombos();
    MessageBox.Show("Zona criada.", "OK", MessageBoxButton.OK, MessageBoxImage.Information);
}
```

A funcionalidade de criar estação necessita de nome de estação, lista de linhas e zonas para que eu possa selecionar. Precisei de fazer comboboxes para ter a lista de zonas e de linhas

Tenho as seguintes comboboxes: cbLinhasDisponiveis e cbZonasDisponiveis

```
<Label Content="Associar à Linha:" Padding="0,2" FontWeight="SemiBold"/>
<ComboBox x:Name="cbLinhasDisponiveis" Height="25" Margin="0,0,0,10" DisplayMemberPath="LetraLinha_

<Label Content="Pertence à Zona Tarifária:" Padding="0,2" FontWeight="SemiBold"/>
<ComboBox x:Name="cbZonasDisponiveis" Height="25" Margin="0,0,0,15" DisplayMemberPath="CodigoZona"
```

A nossa origem dos dados, as listas vêm das coleções globais em App em que a App.lstLinhas e App.lstZonas estão definidas em App.xaml.cs

```
public static List<Pessoa> lstPessoas = new List<Pessoa>();
public static List<CartaoAndante> lstCartoes = new List<CartaoAndante>();
public static List<Zona> lstZonas = new List<Zona>();
public static List<Linha> lstLinhas = new List<Linha>();
public static List<RegistoValidacao> lstRegistos = new List<RegistoValidacao>();
```

Para carregar no UI, criei algumas zonas e listas para popular os dados no App.xaml.cs, adicionei as zonas PRT1, MTS1 e uma linha azul de matosinhos.

```
// Criar duas zonas base
Zona z1 = new Zona { CodigoZona = "PRT1", NomeRegiao = "Porto Centro" };
Zona z2 = new Zona { CodigoZona = "MTS1", NomeRegiao = "Matosinhos" };
lstZonas.Add(z1);
lstZonas.Add(z2);

// criar estacoes e validadores
Estacao trindade = new Estacao { NomeEstacao = "Trindade", Zona = z1 };
trindade.AddValidador(new Validador { Estacao = trindade });
z1.AddEstacao(trindade);

Estacao matosinhos = new Estacao { NomeEstacao = "Matosinhos Sul", Zona = z2 };
matosinhos.AddValidador(new Validador { Estacao = matosinhos });
z2.AddEstacao(matosinhos);

// criar uma linha
Linha linhaAzul = new Linha { LetraLinha = 'A', NomeLinha = "Linha de Matosinhos", Cor =
linhaAzul.AddEstacao(matosinhos);
linhaAzul.AddEstacao(trindade);
lstLinhas.Add(linhaAzul);
```

No construtor de AdminWindow chamo os métodos para já carregar os dados da interface do combobox a partir da lista de linhas e zonas.

```
2 references
private void LoadLinesIntoCombo()
{
    cbLinhasDisponiveis.ItemsSource = null;
    cbLinhasDisponiveis.ItemsSource = App.lstLinhas;
}

2 references
private void LoadZonesIntoCombos()
{
    cbZonasDisponiveis.ItemsSource = null;
    cbZonasCarregamento.ItemsSource = null;
    cbZonasDisponiveis.ItemsSource = App.lstZonas;
    cbZonasCarregamento.ItemsSource = App.lstZonas;
}
```

Busco a lista de linhas e zonas através dos comboboxes e inserir as informações da estação usando o helper

```
private void BtnCriarEstacao_Click(object sender, RoutedEventArgs e)
{
    string nomeEst = txtNomeEstacao.Text.Trim();
    var linha = cbLinhasDisponiveis.SelectedItem as AndanteSys.Models.Linha;
    var zona = cbZonasDisponiveis.SelectedItem as AndanteSys.Models.Zona;

    if (string.IsNullOrEmpty(nomeEst) || linha == null || zona == null)
    {
        MessageBox.Show("Preencha o nome da estação e selecione linha e zona.", "Erro", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var est = new AndanteSys.Models.Estacao { NomeEstacao = nomeEst };
    var eh = new Helpers.EstacaoHelper();
    eh.Insert(est, linha, zona);

    MessageBox.Show("Estação instalada.", "OK", MessageBoxButton.OK, MessageBoxImage.Information);
}
```

Ao abrir a lista na aba de criação de estações, aparecem as zonas que populei no App.xaml.cs

Posso adicionar uma nova linha, adicionei a póvoa de varzim

Adicionei a zona maia

The screenshot shows a web form titled "Configurar Nova Zona". It contains two text input fields: "Código da Zona (Ex: PRT1, MTS1):" with the value "M1" entered, and "Nome da Região / Localidade:" with the value "Maia" entered. Below these fields is a blue button labeled "Adicionar Zona Tarifária".

Se for na estação, consigo ver a nova linha e zona que inseri

The screenshot shows the "Instalar Estação e Validador" form. At the top, there is a navigation bar with "Voltar ao Menu Principal" and tabs for "Emitir Cartão", "Criar Zona", "Criar Linha", and "Criar Estação". The form has two text input fields: "Nome da Estação:" with the value "Pedras Rubras", and "Associar à Linha:" with a dropdown menu showing options "A" and "B". A blue button "Instalar Estação no Sistema" is at the bottom.

This screenshot shows the same "Instalar Estação e Validador" form, but with an additional dropdown menu labeled "Pertence à Zona Tarifária:". This dropdown menu is open, showing a list of options: "PRT1", "MTS1", and "M1". The option "M1" is highlighted with a red rectangular box.

Gestão de cartões

Meti o combobox para seleccionar o tipo de cartão, se é azul ou gold. O zero e o azul e o um e o gold.


```
<ComboBox x:Name="cbTipoCartao" Height="25" Margin="0,0,0,15" SelectedIndex="0">
    <ComboBoxItem Content="Andante Azul (Ocasional)"/>
    <ComboBoxItem Content="Andante Gold (Mensal Pass)"/>
</ComboBox>
```

Implementei a funcionalidade de criar cartões em que é preciso escrever o nome e nif. A partir dos dados, se escolher o tipo de cartão gold, vai ser criado o cartao com o titular da pessoa a partir do nome e nif.

```
private void BtnEmitirCartao_Click(object sender, RoutedEventArgs e)
{
    string nome = txtNomeTitular.Text.Trim();
    string nif = txtNifTitular.Text.Trim();

    var cartaoHelper = new Helpers.CartaoHelper();

    if (cbTipoCartao.SelectedIndex == 1) // Gold
    {
        if (nome.Length == 0 || nif.Length == 0)
        {
            MessageBox.Show("Para emitir um Andante Gold é obrigatório informar Nome e NIF.", "Erro", Message
            return;
        }

        var pessoa = new AndanteSys.Models.Pessoa { Nome = nome, NIF = nif };
        var ph = new Helpers.PessoaHelper();
        ph.Insert(pessoa);

        var gold = new AndanteSys.Models.AndanteGold();
        gold.Titular = pessoa;
        gold.RenovarAssinatura(DateTime.Now.Month);
        cartaoHelper.Insert(gold);
    }
}
```

O caso anterior foi para cartões gold, fiz um else para caso se o combobox for escolhido para cartão azul, vai criar o cartão usando o Pessoa Helper. Os cartões azuis, os titulares são opcionais, as pessoas podem nao meter nome e nif e fica anonimo, mas caso as textboxes forem preenchidas, vai inserir a pessoa na lista, mas é opcional.

```
else // azul pode ser anonimo
{
    var azul = new AndanteSys.Models.AndanteAzul();

    if (!string.IsNullOrEmpty(nome) || !string.IsNullOrEmpty(nif))
    {
        var pessoa = new AndanteSys.Models.Pessoa { Nome = nome, NIF = nif };
        var ph = new Helpers.PessoaHelper();
        ph.Insert(pessoa);
        azul.Titular = pessoa;
    }

    cartaoHelper.Insert(azul);
}
```

O DataGridView chamado dgCartoes está configurado em Views/AdminWindow.xaml

```
<DataGrid Grid.Row="1" x:Name="dgCartoes" AutoGenerateColumns="False" CanUserAddRows="False" IsReadOnly="True">
    <DataGrid.Columns>
        <DataGridTextColumn Header="ID Cartão" Binding="{Binding IdCartao}" Width="110"/>
        <DataGridTextColumn Header="Titular" Binding="{Binding Titular.Nome}" Width="*"/>
        <DataGridTextColumn Header="NIF" Binding="{Binding Titular.NIF}" Width="80"/>
        <DataGridTextColumn Header="Tipo" Binding="{Binding TipoCartao}" Width="70"/>
    </DataGrid.Columns>
</DataGrid>
```

Sempre que a base de cartões muda chamamos RefreshCartoesGrid() que foi uma função que implementei

```
else // azul pode ser anonimo
{
    var azul = new AndanteSys.Models.AndanteAzul();

    if (!string.IsNullOrEmpty(nome) || !string.IsNullOrEmpty(nif))
    {
        var pessoa = new AndanteSys.Models.Pessoa { Nome = nome, NIF = nif };
        var ph = new Helpers.PessoaHelper();
        ph.Insert(pessoa);
        azul.Titular = pessoa;
    }

    cartaoHelper.Insert(azul);
}

// clear input fields
txtNomeTitular.Text = string.Empty;
txtNifTitular.Text = string.Empty;

RefreshCartoesGrid();
MessageBox.Show("Cartao emitido com sucesso.", "Sucesso", MessageBoxButton.OK, MessageBoxImage.Information);
```

No método tem o null que faz com que o null seguido de novo ItemsSource força o DataGrid a reavaliar e a mostrar o conteúdo atual da lista.

```
4 references
private void RefreshCartoesGrid()
{
    dgCartoes.ItemsSource = null;
    dgCartoes.ItemsSource = App.lstCartoes;
}
```

Por exemplo, criei um cartao gold com nome titular tanjil

Criei outro azul e mais um de gold. Após criar os cartões, vai aparecer no datagrid os cartões através da App.IstCartoes

AndanteSys - Painel do Administrador

← Voltar ao Menu Principal

Emitir Cartão Criar Zona Criar Linha Criar Estação

Emitir Novo Título

Nome do Titular:

NIF:

Tipo de Cartão:

Andante Gold (Mensal Pass)

Gravar e Emitir

Base de Títulos Emitidos (Fiscalização)

ID Cartão	Titular	NIF	Tipo
bf3fa83e-f555-4242	tanjil	254234312	GOLD
71fe5c3b-1018-432			AZUL
e821da12-4962-421	ze manel	123123123	GOLD

Operações de Carregamento / Balcão

Qtd. Viag. Zona Cor

Carregar Azul Ativar Mês (Gold)

Agora quero implementar a funcionalidade em que ao interagir e clicar na linha dos dados do cartão, quero que apareça em baixo nas textboxes a zona contratada e quantidade de viagens que tem no cartão que clicar.

ID Cartão	Titular	NIF	Tipo
bf3fa83e-f555-4242	tanjil	254234312	GOLD
71fe5c3b-1018-432			AZUL
e821da12-4962-421	ze manel	123123123	GOLD

Operações de Carregamento / Balcão

Qtd. Viag. Zona Cor

0 PRT

Carregar Azul **Ativar Mês (Gold)**

Para isso, uso o SelectionChanged. Na Interacção com seleção O DataGrid tem SelectionChanged = "DgCartoes_SelectionChanged".

```
<DataGrid Grid.Row="1" x:Name="dgCartoes"
    AutoGenerateColumns="False"
    CanUserAddRows="False"
    IsReadOnly="True"
    SelectionMode="Single"
    Margin="0,0,0,15"
    SelectionChanged="DgCartoes_SelectionChanged">
    <DataGrid.Columns>
        <DataGridTextColumn Header="ID Cartão" Binding="{Binding IdCartao}" Width="110"/>
        <DataGridTextColumn Header="Titular" Binding="{Binding Titular.Nome}" Width="*"/>
        <DataGridTextColumn Header="NIF" Binding="{Binding Titular.NIF}" Width="80"/>
        <DataGridTextColumn Header="Tipo" Binding="{Binding TipoCartao}" Width="70"/>
    </DataGrid.Columns>
</DataGrid>
```

No handler DgCartoes_SelectionChanged obtemos o SelectedItem e mostramos campos específicos (ex.: SaldoViagens, ZonaContratada para AndanteAzul) nas TextBox / ComboBox de carregamento. Se o cartão azul for selecionado, mostra a quantidade de viagens e a zona que está contratada.

```
private void DgCartoes_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    var selected = dgCartoes.SelectedItem as AndanteSys.Models.CartaoAndante;
    if (selected == null)
    {
        txtQtdViagens.Text = "";
        cbZonasCarregamento.SelectedItem = null;
        return;
    }

    if (selected is AndanteSys.Models.AndanteAzul azul)
    {
        // show saldo and contracted zone
        txtQtdViagens.Text = azul.SaldoViagens.ToString();
        if (!string.IsNullOrEmpty(azul.ZonaContratada))
        {
            // select by value (CodigoZona)
            cbZonasCarregamento.SelectedValue = azul.ZonaContratada;
        }
    }
    else
    {
        cbZonasCarregamento.SelectedItem = null;
    }
}
```

Caso clicar num cartao em que o titulo seja gold, acedo a combobox e meto a zona contratada em que coincide com o código da zona do cartao. Na quantidade de viagens vai ficar sempre 0

```
else if (selected is AndanteSys.Models.AndanteGold gold)
{
    txtQtdViagens.Text = "0";
    if (gold.ZonasAutorizadas != null && gold.ZonasAutorizadas.Count > 0)
    {
        // aqui pego o ultimo elemento da lista pq a ultima zona tambem inclui os restos anteriores
        var codigo = gold.ZonasAutorizadas[gold.ZonasAutorizadas.Count - 1].CodigoZona;
        var zonaObj = App.lstZonas.FirstOrDefault(z => z.CodigoZona == codigo);
        cbZonasCarregamento.SelectedItem = zonaObj;
    }
    else
    {
        cbZonasCarregamento.SelectedItem = null;
    }
}
else
{
    txtQtdViagens.Text = "";
    cbZonasCarregamento.SelectedItem = null;
}
```

Também tenho que implementar a funcionalidade de carregar as viagens dos cartões azuis. Se não especificar a zona, por predefinido meti código da zona PRT1. Pego no cartão através do data grid. Na parte de validação da quantidade, meti o nome da variável que ia sair por “qtd” e uso essa variável para carregar as viagens.

```
private void BtnCarregarAzul_Click(object sender, RoutedEventArgs e)
{
    if (dgCartoes.SelectedItem == null)
    {
        MessageBox.Show("Selecione um cartão na grelha.", "Erro", MessageBoxButton.OK, MessageB
        return;
    }

    var cartao = dgCartoes.SelectedItem as AndanteSys.Models.CartaoAndante;
    if (cartao is AndanteSys.Models.AndanteAzul azul)
    {
        if (!int.TryParse(txtQtdViagens.Text.Trim(), out int qtd) || qtd <= 0)
        {
            MessageBox.Show("Quantidade inválida.", "Erro", MessageBoxButton.OK, MessageB
            return;
        }

        var zona = cbZonasCarregamento.SelectedItem as AndanteSys.Models.Zona;
        string codigoZona;
        if (zona != null)
        {
            codigoZona = zona.CodigoZona;
        }
        else
        {
            codigoZona = "PRT1";
        }

        azul.CarregarViagens(qtd, codigoZona);
        cbZonasCarregamento.SelectedValue = codigoZona;
        RefreshCartoesGrid();
    }
}
```

Para o cartão gold renovo ao chamar o método da classe cartao gold, renovar assinatura. Com a mesma coisa do caso do cartao azul, pego o cartao atraves do acesso ao data grid.

```

private void BtnRenovarGold_Click(object sender, RoutedEventArgs e)
{
    if (dgCartoes.SelectedItem == null)
    {
        MessageBox.Show("Selecione um cartão na grelha.", "Erro", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var cartao = dgCartoes.SelectedItem as AndanteSys.Models.CartaoAndante;
    if (cartao is AndanteSys.Models.AndanteGold gold)
    {
        gold.RenovarAssinatura(DateTime.Now.Month);
        RefreshCartoesGrid();
        MessageBox.Show("Assinatura renovada.", "OK", MessageBoxButton.OK, MessageBoxImage.Information);
    }
    else
    {
        MessageBox.Show("O cartão seleccionado não é Andante Gold.", "Erro", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
}

```

Agora consigo ver os textboxes a mudarem apos as alterações dos cartões ao renovar ou carregar viagens. Carreguei um andante azul com 30 viagens e carreguei o outro cartão com 2 viagens da zona contratada MTS1. Ao clicar nas duas linhas de dois cartoes, muda se simultaneamente os dados do textbox

Base de Títulos Emitidos (Fiscalização)

ID Cartão	Titular	NIF	Tipo
79a03923-a21f-4b6			AZUL
de172ffc-eb3e-4ff1			AZUL

Operações de Carregamento / Balcão

Qtd. Viagens (Azul): 30 Zona Contratada: PRT1

Carregar Azul Ativar Mês (Gold)

Base de Títulos Emitidos (Fiscalização)

ID Cartão	Titular	NIF	Tipo
79a03923-a21f-4b6			AZUL
de172ffc-eb3e-4ff1			AZUL

Operações de Carregamento / Balcão

Qtd. Viagens (Azul): 2 Zona Contratada: MTS1

Carregar Azul Ativar Mês (Gold)

Procedi um problema em que não faz sentido ter lista de zonas autorizadas, em vez de ser uma lista, decidi ser apenas uma zona.

Meti um comentário a clarificar:

```
txtQtdViagens.Text = "0";
if (gold.ZonasAutorizadas != null && gold.ZonasAutorizadas.Count > 0)
{
    // agora que tive a pensar, o gold supostamente tem so 1 zona autorizada
    // futuramente devia mudar nos Models do cartao gold, em vez de ser lista de zonas autorizadas
    // ser apenas uma zona autorizada, uma zona inclui as zonas mais pequenas
    // tipo para quem tem MAIA1, pode andar no PRT1
    // mas para quem tem PRT1, não pode andar na MAIA1
    var codigo = gold.ZonasAutorizadas[0].CodigoZona;
    var zonaObj = App.lstZonas.FirstOrDefault(z => z.CodigoZona == codigo);
    cbZonasCarregamento.SelectedItem = zonaObj;
}
```

Fiz algumas mudancas, em vez de `_zonaAutorizada` usar `<list>` uso apenas um

```
0      private int _mesPago;
-      private List<Zona> _zonasAutorizadas;
+      private Zona _zonaAutorizada;
```

```
if (gold.ZonasAutorizadas != null && gold.ZonasAutorizadas.Count > 0)
if (gold.ZonaAutorizada != null)
{
    // agora que tive a pensar, o gold supostamente tem so 1 zona autorizada
    // futuramente devia mudar nos Models do cartao gold, em vez de ser lista de zonas

    // ser apenas uma zona autorizada, uma zona inclui as zonas mais pequenas
    // tipo para quem tem MAIA1, pode andar no PRT1
    // mas para quem tem PRT1, não pode andar na MAIA1
    var codigo = gold.ZonasAutorizadas[0].CodigoZona;
    var codigo = gold.ZonaAutorizada.CodigoZona;
```

Corrigi também o problema em que o cartão não validava se a zona contratada tivesse a estação da zona da estação atual, apenas validava se coincidir as zonas e ignorava as estações, agora verifica se a zona autorizada tem a estação da zona da estação atual. Comentei o primeiro bloco marcado, podia fazer das duas formas mas preferi o ultimo bloco.

```

-         if (_zonaAutorizada != null && _zonaAutorizada.CodigoZona ==
zonaEstacaoAtual.CodigoZona)
+         if (_zonaAutorizada != null)
+         {
-             return true;
+
+             if (_zonaAutorizada.CodigoZona == zonaEstacaoAtual.CodigoZona)
+             {
+                 return true;
+             }
+
+             /* if (_zonaAutorizada.LstEstacao.Any(est => zonaEstacaoAtual.TemEstacao(est)))
+                 return true;
+
+             */
+
+             foreach (var estacao in _zonaAutorizada.LstEstacao)
+             {
+                 if (zonaEstacaoAtual.TemEstacao(estacao))
+                 {
+                     return true;
+                 }
+             }
+         }

```

Visualização de registos

Adicionei a função de ver registos e ver o id e nif para conseguir copiar.

```

<StackPanel Orientation="Horizontal" Margin="0, 0, 15, 0" VerticalAlignment="Center">
<Label Content="ID Cartão:" VerticalAlignment="Center" Padding="0, 0, 5, 0"/>
<TextBox x:Name="txtSelecioneIdCartao" Width="250" Height="25" IsReadOnly="True"/>
</StackPanel>

<StackPanel Orientation="Horizontal" VerticalAlignment="Center">
<Label Content="NIF Titular:" VerticalAlignment="Center" Padding="0, 0, 5, 0"/>
<TextBox x:Name="txtSelecioneNif" Width="160" Height="25" IsReadOnly="True"/>
</StackPanel>
</StackPanel>

```

Na função de DgCartoes_SelectionChanged, busco o cartão selecionado que esta como selected, a partir daí acedo os atributos

```

txtSelecioneIdCartao.Text = selected.IdCartao.ToString();
if (selected.Titular != null)
{
    txtSelecioneNif.Text = selected.Titular.NIF;
}
else
{
    txtSelecioneNif.Text = string.Empty;
}

```


Base de Títulos Emitidos (Fiscalização)

ID Cartão	Titular	NIF	Tipo
24bfabdc-4ba8-497	Carlos Silva	123456789	GOLD
7db4552d-0c4a-42	Ana Pereira	987654321	AZUL
8a08d630-d061-49			AZUL
9002f5e1-3003-48c	João Sousa	555444333	GOLD

Operações de Carregamento / Balcão

Qtd. Viagens (A) Zona Contratac
0 PRT1

Carregar Azul Ativar Mês (Gold) Ver Registos

ID Cartão: 24bfabdc-4ba8-4973-9fcd-4f1eeeb77b1c NIF Titular: 123456789

Esta é a janela de registos

```
<Window x:Class="AndanteSys.Views.RegistosWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Histórico de Registos de Validação" Height="450" Width="700" WindowStartupLocation="CenterOwner">

    <Grid Margin="10">
        <TextBlock Text="Histórico de Validações" FontSize="18" FontWeight="Bold" Margin="0,0,0,10"/>
        <DataGrid x:Name="dgRegistos" Margin="0,40,0,0" AutoGenerateColumns="False" IsReadOnly="True" CanUserAddRows="False">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Data / Hora" Binding="{Binding DataValidacao}" Width="160"/>
                <DataGridTextColumn Header="Resultado" Binding="{Binding Sucesso}" Width="80"/>
                <DataGridTextColumn Header="Zona" Binding="{Binding ZonaValidada}" Width="80"/>
                <DataGridTextColumn Header="Titular" Binding="{Binding Cartao.Titular.Nome}" Width="*" />
                <DataGridTextColumn Header="Tipo Cartão" Binding="{Binding Cartao.TipoCartao}" Width="100"/>
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Window>
```

Uso o helper de registos para retornar a lista de registos e meter no datagrid.

```
1 reference
public RegistosWindow()
{
    InitializeComponent();
    LoadRegistos();
}

1 reference
private void LoadRegistos()
{
    var rh = new RegistoValidacaoHelper();
    dgRegistos.ItemsSource = null;
    dgRegistos.ItemsSource = rh.ListarHistorico();
}
```

Histórico de Registos de Validação

Histórico de Validações

Data / Hora	Resultado	Zona	Titular	Tipo Cartão
6/21/2026 8:54:05 PM	True	PRT1	Carlos Silva	GOLD
6/21/2026 8:54:05 PM	True	PRT1	Carlos Silva	GOLD
6/21/2026 8:54:18 PM	True	PRT1	Carlos Silva	GOLD
6/21/2026 8:54:01 PM	False	MTS1	Carlos Silva	GOLD
6/21/2026 8:53:46 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:45 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:45 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:45 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:28 PM	False	PRT1	João Sousa	GOLD
6/21/2026 8:53:28 PM	False	PRT1	João Sousa	GOLD
6/21/2026 8:53:25 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:25 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:25 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:25 PM	True	MAI1	João Sousa	GOLD
6/21/2026 8:53:19 PM	False	PRT1	João Sousa	GOLD
6/21/2026 8:54:01 PM	False	MTS1	Carlos Silva	GOLD
6/21/2026 8:54:19 PM	True	PRT1	Carlos Silva	GOLD

Janela Cliente

Autenticação

Altere a logica na parte da funcionalidade do evento de clicar no botao login, pesquisa pelo GUID id ou pelo NIF primeiro

```
private void BtnLogin_Click(object sender, RoutedEventArgs e)
{
    string input = txtLoginInput.Text.Trim();
    if (input.Length == 0)
    {
        MessageBox.Show("Por favor, introduza um NIF ou ID válido.", "Aviso", MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }

    // tenta encontrar o cartao
    AndanteSys.Models.CartaoAndante cartaoEncontrado = null;
    // pesquisa pelo GUID
    cartaoEncontrado = App.lstCartoes.FirstOrDefault(c => c.IdCartao.ToString().Equals(input, StringComparison.OrdinalIgnoreCase));
    if (cartaoEncontrado == null)
    {
        // tenta os primeiros 8 caracteres
        cartaoEncontrado = App.lstCartoes.FirstOrDefault(c => c.IdCartao.ToString().StartsWith(input, StringComparison.OrdinalIgnoreCase));
    }

    if (cartaoEncontrado == null)
    {
        // pesquisa pelo NIF
        var pessoa = App.lstPessoas.FirstOrDefault(p => p.NIF == input);
        if (pessoa != null)
        {
            cartaoEncontrado = App.lstCartoes.FirstOrDefault(c => c.Titular != null && c.Titular.IdPessoa == pessoa.Id);
        }
    }

    if (cartaoEncontrado == null)
    {
        MessageBox.Show("Nenhum cartão ou titular encontrado com essa identificação.", "Aviso", MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }
}
```

Após a autenticação ser sucedida, mostro na UI as informações do cartão com o acesso dos atributos

```
string tipo = cartaoEncontrado.TipoCartao ?? "Desconhecido";
string nomeTit;
if (cartaoEncontrado.Titular != null)
{
    nomeTit = cartaoEncontrado.Titular.Nome;
}
else
{
    nomeTit = "Anónimo";
}

lblStatusSessao.Text = $"Sessão Ativa: {nomeTit} ({tipo})";

if (cartaoEncontrado is AndanteSys.Models.AndanteAzul azul)
{
    lblSaldoSessao.Text = $"Saldo: {azul.SaldoViagens} viagem(ns) | Zona: {azul.ZonaContratada}";
}
else if (cartaoEncontrado is AndanteSys.Models.AndanteGold gold)
{
    string zonas;
    if (gold.ZonaAutorizada != null)
    {
        zonas = gold.ZonaAutorizada.CodigoZona;
    }
    else
    {
        zonas = "Nenhuma";
    }

    lblSaldoSessao.Text = $"Assinatura mês: {gold.MesPago} | Zona: {zonas}";
}
else
{
    lblSaldoSessao.Text = "Cartão sem informação adicional.";
}
```

Tenho este cartão, copiei o Guid id do cartao

Base de Títulos Emitidos (Fiscalização)

ID Cartão	Titular	NIF	Tipo
563ea9a6-66ce-4c6	Carlos Silva	123456789	GOLD
c911ae7f-1b0f-4f35	Ana Pereira	987654321	AZUL
96a84d8f-2b1e-4b6			AZUL
76604264-7b7f-44f	João Sousa	555444333	GOLD

Operações de Carregamento / Balcão

Qtd. Viagem: Zona Contr:

ID Cartão: NIF Titular:

E aparece aqui as informações da sessão a partir do cartão

Modifiquei a UI para ter melhor acessibilidade para o utilizador em que consegue ver todas as zonas, quais estações tem

Ver zonas e estações

Adicionei um treeview para cada zona, ver as estações

```

75 +         <Border Grid.Column="1" BorderBrush="#DDD" BorderThickness="1" CornerRadius="5"
      Padding="10" Margin="15,0,0,0" Background="#FBFBFB">
76 +             <StackPanel>
77 +                 <TextBlock Text="Zonas e Estações" FontSize="14" FontWeight="Bold"
      Foreground="#007ACC" Margin="0,0,0,8"/>
78 +                 <TextBlock Text="Clique numa estação para seleccioná-la." FontSize="11"
      Foreground="#666" Margin="0,0,0,10"/>
79 +                 <ScrollViewer Height="230" VerticalScrollBarVisibility="Auto">
80 +                     <TreeView x:Name="tvZonas" />
81 +                 </ScrollViewer>
82             </StackPanel>
83         </Border>
84 +     </StackPanel>
85 + </Grid>

```

Para cada zona, acedo à lista de estações e implemento no UI

```
var tree = this.FindName("tvZonas") as TreeView;
if (tree == null) return;
tree.Items.Clear();
if (App.lstZonas == null) return;

foreach (var zona in App.lstZonas)
{
    var header = $"{zona.CodigoZona} - {zona.NomeRegiao} ({zona.LstEstacao.Count} estações)";
    var tzi = new System.Windows.Controls.TreeViewItem() { Header = header, Tag = zona };

    foreach (var est in zona.LstEstacao)
    {
        var child = new System.Windows.Controls.TreeViewItem() { Header = est.NomeEstacao, Tag = est };
        tzi.Items.Add(child);
    }

    tree.Items.Add(tzi);
}
```

Validar estações

Tive um problema em que os cartões com zona contratada da maia, não conseguia poder validar as zonas que pertencem a zona da porto, supostamente, a zona de maia ou de matosinhos consegue validar a zona do porto. Portanto foi um erro em que tive que resolver e tive que mudar a lógica da validação, mudei a validação de zona por estação.

```
src/AndanteSys/Models/CartaoAndante.cs
@@ -51,7 +51,7 @@ protected CartaoAndante()
1  51      _titular = new Pessoa();
2  52  }
3  53
4  - public abstract bool ValidarViagem(Zona zonaEstacaoAtual);
5  + public abstract bool ValidarViagem(Estacao estacaoAtual);
6  55
7  + public override string ToString()
8  56  {
9  57  }
```

Faço a verificação para buscar a zona do contrato e verificar se inclui a estação atual no cartão azul.

```
+
+      // Se for zona de sobreposição, vai buscar a zona do contrato e vê se ela inclui esta
+      estação
+
+      var zonaDoContrato = App.lstZonas.FirstOrDefault(z => z.CodigoZona == _zonaContratada);
+      if (zonaDoContrato != null && zonaDoContrato.TemEstacao(estacaoAtual))
+      {
+          _saldoViagens--;
+          return true;
+      }
```

Aqui no cartao gold

```
// 2. Se for outra zona, basta ver se a zona do contrato conhece esta estação específica
if (_zonaAutorizada.TemEstacao(estacaoAtual))
{
    return true;
}
```

Agora para validar a viagem, em vez de usar a zona, uso a estação

```
src/AndanteSys/Views/ClienteWindow.xaml.cs
@@ -139,7 +139,7 @@ private void BtnValidar_Click(object sender, RoutedEventArgs)
139 139         else
140 140         {
141 141             // fallback: validate directly against station zone
142 142             resultado = cartao.ValidarViagem(estacao.Zona);
142 142             resultado = cartao.ValidarViagem(estacao);
143 143
144 144             // create and persist registo manually
145 145             var novo = new AndanteSys.Models.RegistoValidacao();
```

```
private void BtnValidar_Click(object sender, RoutedEventArgs e)
{
    var cartao = panelViagem.Tag as AndanteSys.Models.CartaoAndante;
    if (cartao == null)
    {
        MessageBox.Show("Nenhum cartão válido na sessão. Faça login novamente.", "Erro", MessageBoxButton.OK);
        return;
    }

    var estacao = cbEstacoes.SelectedItem as AndanteSys.Models.Estacao;
    if (estacao == null)
    {
        MessageBox.Show("Selecione uma estação de embarque.", "Aviso", MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }

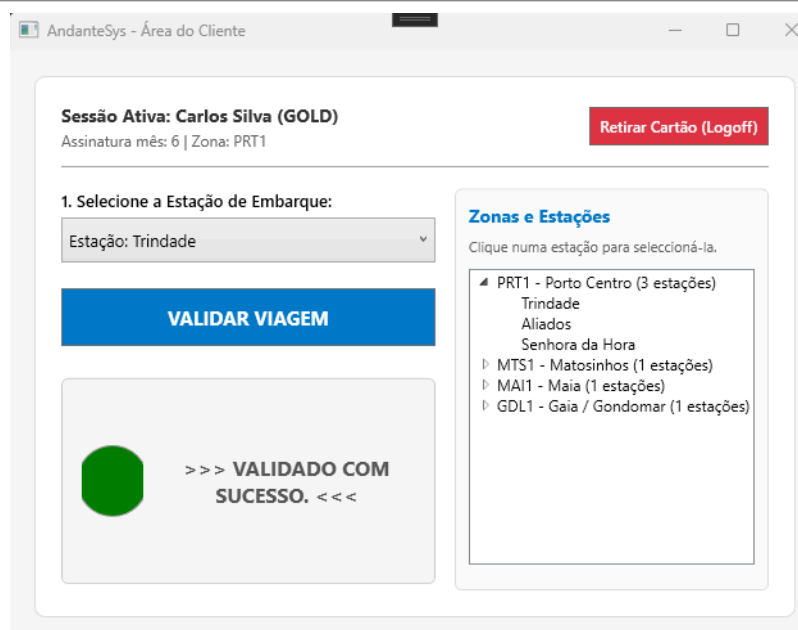
    bool resultado = false;
    if (estacao.Validador != null)
    {
        resultado = estacao.Validador.ProcessarLeitura(cartao);
    }
}
```

```
1 reference
public bool ProcessarLeitura(CartaoAndante c)
{
    if (!_statusAtivo || c == null || _estacao == null || _estacao.Zona == null)
        return false;

    bool sucessoViagem = c.ValidarViagem(_estacao);

    RegistoValidacao novoRegisto = new RegistoValidacao();
    novoRegisto.Sucesso = sucessoViagem;
    novoRegisto.ZonaValidada = _estacao.Zona.CodigoZona;
    novoRegisto.Cartao = c;
    HistoricoGlobal.Add(novoRegisto);

    AndanteSys.Helpers.RegistoValidacaoHelper registoHelper = new AndanteSys.Helpers.RegistoValidacaoHelper();
    registoHelper.Insert(novoRegisto);
    return sucessoViagem;
}
```



Conclusão

Com a conclusão deste projeto, foi possível desenhar e implementar com sucesso o sistema de gestão de rede de metropolitano e validação de títulos de transporte AndanteSys. O desenvolvimento focou-se no cumprimento dos requisitos base e essenciais estipulados para o funcionamento da aplicação, garantindo uma interface gráfica funcional em WPF (com as janelas MainWindow, AdminWindow e ClientWindow) e a persistência de dados em memória através de padrões arquiteturais organizados.

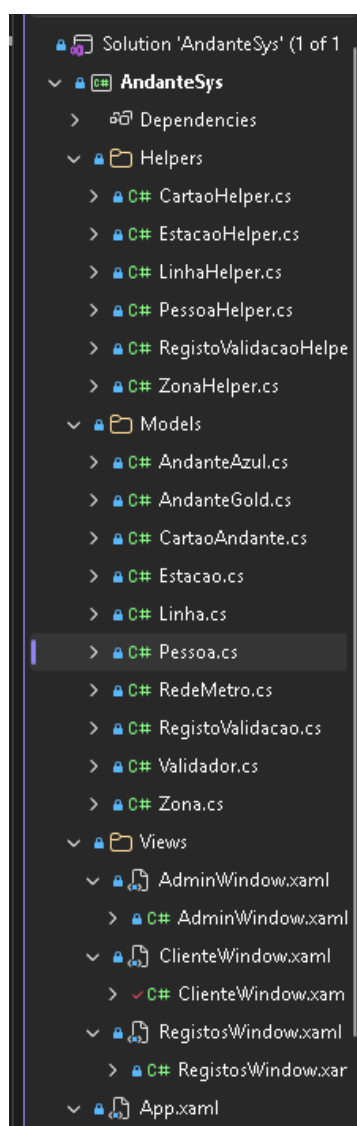
No entanto, devido às restrições de tempo para a entrega do projeto, algumas funcionalidades adicionais e operações mais avançadas não foram implementadas, tais como a operação de remoção de dados (as rotinas de "Apagar" linhas, estações ou cartões), priorizando-se ao invés disso a estabilização e a fiabilidade do núcleo do sistema.

Durante o processo de desenvolvimento e simulação, surgiram desafios que exigiram refatorizações críticas na arquitetura inicialmente pensada. Destaca-se a alteração da estrutura da classe Estação, que foi simplificada de uma coleção `List<Validador>` para apenas um único objeto `Validador`, aproximando o modelo

da realidade física de uma estação. Adicionalmente, verificou-se a necessidade de reformular a lógica de validação de passes e viagens na rede: a assinatura do método foi alterada para receber o objeto `Estacao` em vez da `Zona`, corrigindo em definitivo os problemas de validação indevida em zonas contíguas com estações partilhadas (como o Porto e a Maia).

Em suma, apesar das limitações de tempo que impediram a inclusão de mecânicas CRUD completas, o projeto atingiu os objetivos pretendidos, resultando numa aplicação robusta, consistente com as regras de negócio do Metro do Porto e estruturalmente bem conseguida.

Portanto a estrutura final dos ficheiros foi esta:



Modelo de classes atualizado

O diagrama atualizado

